

// Jivops

Redis

Guía completa — 20 lecciones

Fundamentos y estructuras · Patrones y rendimiento · Persistencia,
replicación y alta disponibilidad · Seguridad y producción

Contenido

Módulo 1: Fundamentos y estructuras de datos

- Lección 01 — Qué es Redis y su modelo de ejecución
- Lección 02 — Estructuras de datos y cuándo usar cada una
- Lección 03 — Expiración de claves y TTL
- Lección 04 — Políticas de evicción y maxmemory
- Lección 05 — Transacciones, WATCH y scripts Lua

Módulo 2: Patrones de uso y rendimiento

- Lección 06 — Patrón cache-aside y avalancha de caché
- Lección 07 — Diseño de claves — SCAN, UNLINK, big keys y hot keys
- Lección 08 — Pipelining y latencia de red
- Lección 09 — Pub/Sub frente a Streams
- Lección 10 — Bloqueos distribuidos con Redis

Módulo 3: Persistencia, replicación y alta disponibilidad

- Lección 11 — Persistencia — RDB y AOF
- Lección 12 — Replicación asíncrona y sus consecuencias
- Lección 13 — Redis Sentinel — alta disponibilidad
- Lección 14 — Redis Cluster — hash slots y hash tags
- Lección 15 — Memoria, fork y copy-on-write

Módulo 4: Seguridad y producción

- Lección 16 — Seguridad de red y autenticación
- Lección 17 — ACL y comandos peligrosos
- Lección 18 — TLS y cifrado
- Lección 19 — Monitoreo — INFO, SLOWLOG y métricas clave
- Lección 20 — Proyecto final: Redis listo para producción

Módulo 1: Fundamentos y estructuras de datos

Lección 01 — Qué es Redis y su modelo de ejecución

Es un almacén clave-valor en memoria con estructuras de datos avanzadas y latencias de submilisegundo; ejecuta los comandos en un único hilo principal, así que un comando lento bloquea a los demás clientes, y los datos pueden persistirse en disco.

Como los comandos se ejecutan en un único hilo, un comando sobre una colección enorme bloquea a los demás clientes hasta que termina; se evita con comandos incrementales o dividiendo las colecciones.

Lección 02 — Estructuras de datos y cuándo usar cada una

Strings para valores y contadores, hashes para objetos con campos, listas para colas simples, sets para elementos únicos, sorted sets para rankings ordenados por puntuación y streams para registros de eventos.

La estructura ya ofrece operaciones como incrementar un campo o consultar un rango ordenado en el propio servidor, así que se evita transferir datos y recalcular en la aplicación, con operaciones atómicas y eficientes.

Lección 03 — Expiración de claves y TTL

Una clave puede expirar con SET ... EX o EXPIRE; Redis la elimina de forma pasiva al acceder a ella y de forma activa muestreando claves con TTL, así que una clave expirada puede ocupar memoria un breve tiempo; PERSIST quita la expiración.

SET sustituye el valor y descarta el TTL anterior salvo que se use KEeptTL; la clave pasa a ser permanente y puede acumularse sin límite, así que hay que volver a indicar la expiración al escribir.

Lección 04 — Políticas de evicción y maxmemory

Depende de la política: noeviction (la predeterminada) rechaza las escrituras con error, allkeys-lru y allkeys-lfu expulsan claves de todo el conjunto (típico en caché) y las volatile-* solo expulsan claves con TTL.

Al llenarse la memoria expulsa claves aunque sean datos que no se pueden regenerar; una caché admite perder entradas pero una base de datos no, así que para datos críticos conviene noeviction y dimensionar la memoria.

Lección 05 — Transacciones, WATCH y scripts Lua

MULTI/EXEC ejecuta los comandos en cola de forma secuencial y sin intercalar a otros clientes, pero no hay rollback: si uno falla al ejecutarse, los demás se ejecutan igualmente; WATCH añade bloqueo optimista abortando EXEC si una clave vigilada cambió, y un script Lua es atómico como un solo comando.

Se ejecuta de forma atómica en el hilo principal y ningún otro comando se atiende mientras dura, así que un script lento detiene todas las peticiones; deben ser breves y acotados.

Módulo 2: Patrones de uso y rendimiento

Lección 06 — Patrón cache-aside y avalancha de caché

La aplicación consulta primero Redis y, si no está (miss), lee de la base de datos y guarda el resultado con TTL; al modificar el dato se invalida o actualiza la clave; para evitar que muchos fallos simultáneos golpeen la base al expirar claves populares, se añade jitter al TTL o se serializa la recarga.

Expiran en el mismo instante y todas las peticiones fallan a la vez hacia la base de datos, que recibe de golpe una carga para la que no está dimensionada; un jitter reparte las expiraciones en el tiempo.

Lección 07 — Diseño de claves — SCAN, UNLINK, big keys y hot keys

Usar nombres con prefijo (usuario:123:perfil), recorrer con SCAN (incremental) en lugar de KEYS * (bloqueante), borrar colecciones grandes con UNLINK y vigilar las big keys y hot keys, que concentran memoria o tráfico en una sola clave o nodo.

Todo el tráfico de esa clave va al nodo que posee su slot y no se reparte; ese nodo se satura mientras los demás están ociosos, y añadir nodos no ayuda porque la clave sigue en un único lugar.

Lección 08 — Pipelining y latencia de red

Envía varios comandos seguidos sin esperar cada respuesta, reduciendo el coste de ida y vuelta de red; no es atómico (a diferencia de MULTI) y conviene enviar lotes de tamaño acotado, porque el servidor acumula las respuestas en memoria.

La latencia de red domina el tiempo de cada comando; con pipelining se paga una vez por lote en lugar de por comando, ya que se envían muchos comandos en un solo viaje y el servidor ejecuta rápido.

Lección 09 — Pub/Sub frente a Streams

Pub/Sub entrega mensajes solo a los suscriptores conectados en ese momento y no los guarda; los Streams son un registro persistente donde los consumidores, agrupados en consumer groups, leen, confirman con XACK y pueden reprocesar mensajes pendientes, por lo que se adaptan mejor a colas fiables.

Si el suscriptor está desconectado o falla, el mensaje se pierde sin que nadie lo guarde, y no hay confirmación ni reintento de entrega; los Streams con grupos permiten reprocesar lo pendiente.

Lección 10 — Bloqueos distribuidos con Redis

Se toma con SET clave token NX PX ttl (token único, solo se crea si no existe y caduca) y se libera comprobando de forma atómica, con un script, que el token sigue siendo el propio; aun así, si el trabajo dura más que el TTL el lock expira y dos clientes pueden creerse dueños, por lo que para exclusión estricta hace falta un fencing token o un sistema de consenso.

El lock original pudo expirar y otro cliente adquirirlo, y un DEL ciego eliminaría el lock del nuevo dueño; el token único permite comprobar que el lock sigue siendo el propio, y la comprobación y el borrado deben ser atómicos.

Módulo 3: Persistencia, replicación y alta disponibilidad

Lección 11 — Persistencia — RDB y AOF

RDB guarda instantáneas periódicas compactas con reinicio rápido, pero se pueden perder los cambios desde la última; AOF registra cada escritura y con `appendfsync everysec` (habitual) se pierde como máximo en torno a un segundo; pueden combinarse, y sin persistencia Redis funciona como caché pura.

Las instantáneas se hacen a intervalos, y los cambios posteriores a la última se pierden si el servidor cae; AOF registra las escrituras de forma continua y reduce la ventana de pérdida.

Lección 12 — Replicación asíncrona y sus consecuencias

Es asíncrona: el primario responde al cliente antes de que las réplicas apliquen el cambio, así que un failover puede perder escrituras ya confirmadas y las lecturas en réplicas pueden estar desactualizadas; WAIT espera confirmaciones de réplicas pero no convierte el sistema en fuertemente consistente.

La réplica recibe los cambios con un pequeño retraso y aún no ha aplicado la última escritura, porque la replicación asíncrona no espera a las réplicas antes de confirmar; hay que aceptar lecturas ligeramente desactualizadas o leer del primario.

Lección 13 — Redis Sentinel — alta disponibilidad

Monitoriza primario y réplicas, detecta fallos por acuerdo entre varios sentinels (quórum), promueve una réplica y avisa a los clientes del nuevo primario; se despliegan al menos 3 y da alta disponibilidad sobre un único conjunto de datos, sin repartir los datos entre nodos.

Un solo Sentinel podría tener un problema de red propio y creer erróneamente que el primario está caído; el acuerdo de la mayoría evita falsos positivos y failovers innecesarios, y también se necesita mayoría para autorizar el failover.

Lección 14 — Redis Cluster — hash slots y hash tags

Reparte las claves en 16384 hash slots entre varios primarios ($\text{slot} = \text{CRC16 de la clave módulo } 16384$), con réplicas para failover; los clientes siguen redirecciones MOVED, y los comandos con varias claves solo funcionan si todas están en el mismo slot, lo que se logra con hash tags como `{usuario1}`.

Las claves deben estar en el mismo slot, y por tanto en el mismo nodo, para ejecutarse juntas; el hash tag fuerza que varias claves calculen el slot con la misma parte del nombre, y sin él caen en slots distintos y el comando se rechaza.

Lección 15 — Memoria, fork y copy-on-write

Al guardar RDB o reescribir el AOF, Redis hace un fork con copy-on-write: con muchas escrituras la memoria usada puede crecer mucho, y el propio fork puede causar picos de latencia en conjuntos grandes; por eso no se asigna toda la RAM a `maxmemory` y se recomienda desactivar las Transparent Huge Pages.

Durante el fork las páginas modificadas se copian y el consumo real puede superar el tamaño de los datos; sin margen, el sistema podría quedarse sin memoria y matar el proceso, y la memoria extra cubre el copy-on-write y los buffers.

Módulo 4: Seguridad y producción

Lección 16 — Seguridad de red y autenticación

Redis está pensado para redes de confianza: no debe exponerse a internet, se limita con bind, firewall y protected-mode, y se exige autenticación (ACL o requirepass); una instancia abierta y sin contraseña permite a un atacante leer, borrar o modificar todos los datos e incluso abusar de la configuración.

Los escaneos de red encuentran servicios en cualquier puerto y la seguridad por oscuridad no impide el acceso; sin autenticación, cualquiera que llegue al puerto puede ejecutar comandos, así que la protección real es no exponerlo y exigir autenticación.

Lección 17 — ACL y comandos peligrosos

Desde Redis 6 permiten crear usuarios con contraseña, patrones de claves permitidos (~app:*) y categorías de comandos (+@read, -@dangerous), aplicando el mínimo privilegio; conviene restringir comandos como FLUSHALL, CONFIG, DEBUG o KEYS a cuentas administrativas.

Si se compromete, solo puede leer o modificar sus propias claves y no las de otros servicios; restringir comandos peligrosos evita borrados masivos o cambios de configuración, y el mínimo privilegio reduce el impacto de un error o una intrusión.

Lección 18 — TLS y cifrado

Redis 6 y posteriores admiten TLS (puerto TLS, certificados y opcionalmente autenticación mutua) para cifrar el tráfico entre clientes, réplicas y nodos de cluster; Redis no cifra por sí solo los datos en reposo, así que los archivos RDB y AOF requieren cifrado de disco o de sistema de archivos.

TLS protege los datos mientras viajan por la red, no los archivos ya guardados; el archivo RDB contiene los datos en claro en disco, así que hace falta cifrado de disco o de sistema de archivos para el reposo.

Lección 19 — Monitoreo — INFO, SLOWLOG y métricas clave

INFO ofrece métricas clave: memoria usada y fragmentación, claves expulsadas (evicted_keys), aciertos y fallos de caché, clientes conectados y estado de la replicación; SLOWLOG registra los comandos lentos (mide el tiempo de ejecución, no el de red) y se suele exportar a Prometheus con un exporter.

Muchas expulsiones indican que la memoria es insuficiente y pueden bajar la tasa de aciertos, lo que aumenta la carga sobre la base de datos de origen; juntas muestran si la caché está bien dimensionada.

Lección 20 — Proyecto final: Redis listo para producción

Estructuras y claves bien diseñadas sin comandos bloqueantes, política de evicción acorde al uso, persistencia elegida según la pérdida tolerable, Sentinel o Cluster con réplicas, ACL y TLS sin exposición a internet, margen de memoria para el fork y monitoreo de latencia, evicciones y replicación.

Intenta mencionar al menos dos pilares: estructuras y claves bien diseñadas, evicción y persistencia acordes al uso, Sentinel o Cluster con réplicas, ACL y TLS sin exposición a internet, o monitoreo de latencia y evicciones.